DATAWISE AI

DataWise AI: Research Whitepaper

An Adaptive Multi-Agent Platform for Autonomous Data Science & Machine Learning Engineering

Document Type:	Comprehensive Research Whitepaper
Target Audience:	Machine Learning Researchers, System Architects, Evaluators
Document Version:	v1.0.0 (Engineering Release)
Date of Issue:	October 27, 2026
Repository Status:	Verified Codebase & Production Architecture
Orchestration:	LangGraph 16-Agent Autonomous Architecture

Architectural Foundation: Deterministic ML execution engine paired with autonomous multi-agent reasoning, strict data leakage boundaries, and human-in-the-loop validation.

Abstract

Modern machine learning engineering workflows are plagued by severe fragmentation across dataset profiling, data quality assessment, preprocessing, feature engineering, model selection, hyperparameter optimization, and deployment verification. While conventional Automated Machine Learning (AutoML) frameworks rely on rigid, black-box search heuristics, they consistently fail to provide explanatory context, detect complex data leakage, incorporate business constraints, or provide interactive oversight. This paper introduces **DataWise AI**, an end-to-end autonomous data science platform built on an orchestrated multi-agent architecture utilizing LangGraph. DataWise AI decouples probabilistic large language model (LLM) reasoning from deterministic computation: all statistics, data transformations, cross-validations, and model evaluations are strictly executed by Python tools (scikit-learn, XGBoost, LightGBM, Optuna), while agents reason over structured summaries. The platform introduces a structured two-prompt problem definition interface, an adaptive hyperparameter workbench, an 18-module resilience and error recovery subsystem, and reproducible pipeline serialization. We analyze the complete architecture, inspect the verified implementation, detail leak-prevention mechanisms, and formalize an empirical research roadmap.

1. Introduction and Platform Motivation

Building production-grade machine learning pipelines requires multidisciplinary expertise spanning data engineering, statistical diagnostics, feature synthesis, algorithmic trade-off analysis, and operations. In typical enterprise workflows, data scientists spend upwards of 70% of engineering cycles on data cleaning, schema validation, and repetitive pipeline boilerplate. Existing automated solutions present a false dichotomy: fully automated 'black-box' tools that strip developers of control and obscure data leakage, versus manual Jupyter notebook workflows that are slow, error-prone, and difficult to reproduce.

DETERMINISTIC EXECUTION ISOLATION

Critical Architectural Principle: Large Language Models must NEVER directly perform numerical computations, statistical calculations, or dataset slicing. DataWise AI strictly confines LLMs to strategic planning, tool selection, interpretation of deterministic outputs, and human communication.

2. Problem Formulation and Measurable Objectives

DataWise AI resolves six core failure modes observed in modern data science teams:

- **Data Leakage Vulnerabilities:** Preprocessing transformations fitted across entire datasets prior to cross-validation splits, resulting in overly optimistic validation scores and catastrophic production degradation.
- **Opaque Hyperparameter Optimization:** Blind grid search wasting cloud compute budgets without adapting search spaces based on dataset properties (e.g., sparsity, high-cardinality, class imbalance).
- **Silent Pipeline Failures:** Schema mismatches, unhandled outliers, and missing category levels in unseen inference data causing uncaught runtime exceptions.
- **Artifact Incoherence:** Disconnected deployment artifacts where trained weights, preprocessing transformers, and feature mappings are stored without cryptographic fingerprinting or version tracking.

Objective Dimension	Target Capability	Implementation Verification
Functional	Autonomous E2E workflow from raw upload to deployment bundle	Verified via 16 numbered LangGraph agents
Engineering	Strict separation of probabilistic LLM and deterministic ML	Implemented via tool abstraction layer
ML Quality	Strict leakage prevention (train-only fit of transformers)	Verified by LeakageAgent and automated tests
Resilience	Autonomous recovery from API rate limits and memory limits	18-module error recovery engine in backend/recovery
Security	Multi-tier authentication and strict project isolation	Clerk OAuth + Supabase + Local JWT, RBAC

3. Full-Stack System Architecture

DataWise AI is architected as an asynchronous distributed system comprising a high-responsiveness frontend, a FastAPI backend, an orchestrated agent network, and a multi-backend storage layer:

Layer	Technology Stack	Role in Platform Architecture
Frontend UI	Vite, React 18, TypeScript, Tailwind CSS, Lucide	Responsive data science workbench, interactive chat, visualizer
API Gateway	FastAPI, Uvicorn, Pydantic v2, Python 3.11+	Asynchronous REST endpoints, WebSockets, rate limiting
Orchestrator	LangGraph, StateGraph, TypedDict Shared State	State machine managing multi-agent execution, routing, rollbacks
ML Execution	scikit-learn, XGBoost, LightGBM, Optuna	Deterministic computation, cross-validation, hyperparameter tuning
LLM Router	Gemini, Groq, Cloudflare, Ollama, OpenAI	Multi-provider resilience with automatic fallbacks and circuit breaker
Database	SQLAlchemy 2.0, PostgreSQL / SQLite (37 models)	Relational persistence of users, projects, runs, metrics, audit logs
Object Storage	MinIO S3 / Local File Storage / Supabase	Secure bucket storage for datasets, pickled pipelines, and notebooks
Cache & Limiter	Redis (with in-memory fallback)	Distributed sliding window rate limiter, run caching

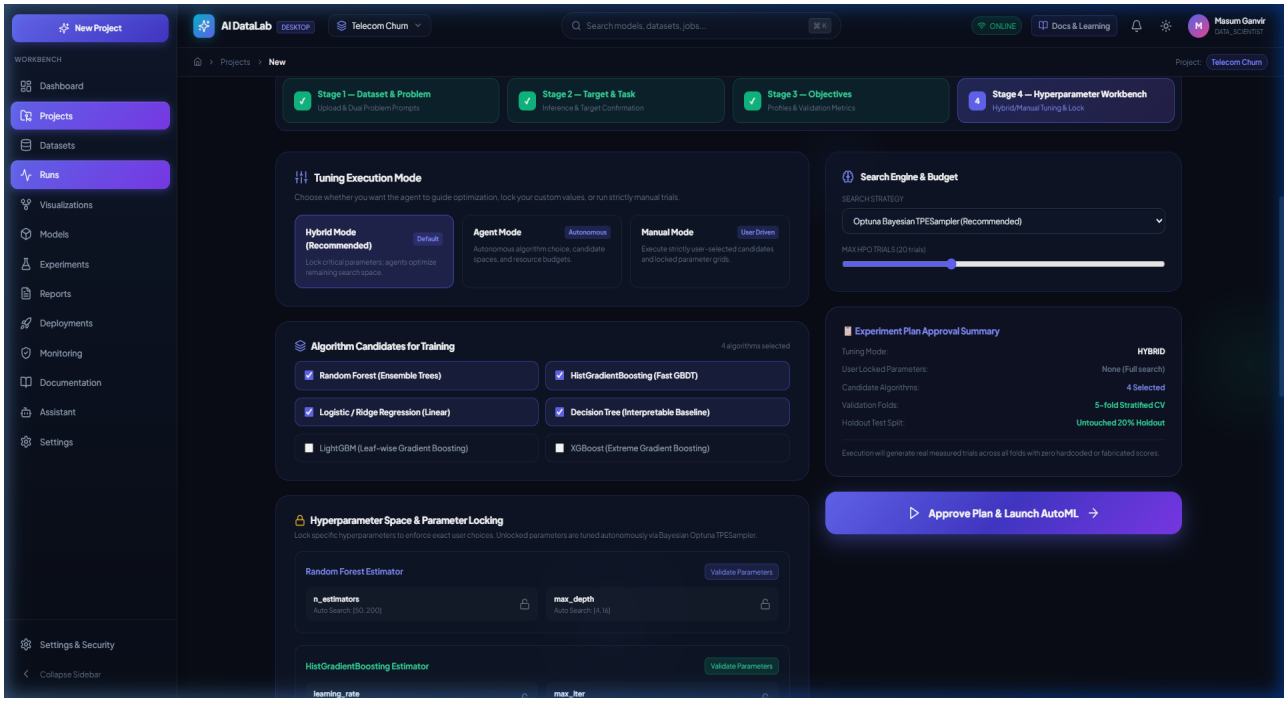


Figure: DataWise AI Stage 4 Interactive Workbench and Agent Chat Interface

4. The Two-Prompt Problem Definition Protocol

To eliminate ambiguity in automated problem formulation, DataWise AI implements a dual-prompt user onboarding protocol:

- **Prompt 1 (Problem Domain):** *'What business problem are you trying to solve?'* — The user expresses their operational domain, objective, and qualitative success criteria.
- **Prompt 2 (Solution Constraints):** *'What solution do you want, and how should it work?'* — The user defines technical constraints, runtime budgets, target variables, and interpretability preferences.

The **UserIntentAgent** parses these prompts alongside dataset schema fingerprints, constructing a structured JSON specification containing the inferred problem type (binary classification, multi-class, regression, time series), the candidate target column, evaluation metrics, and optimization constraints.

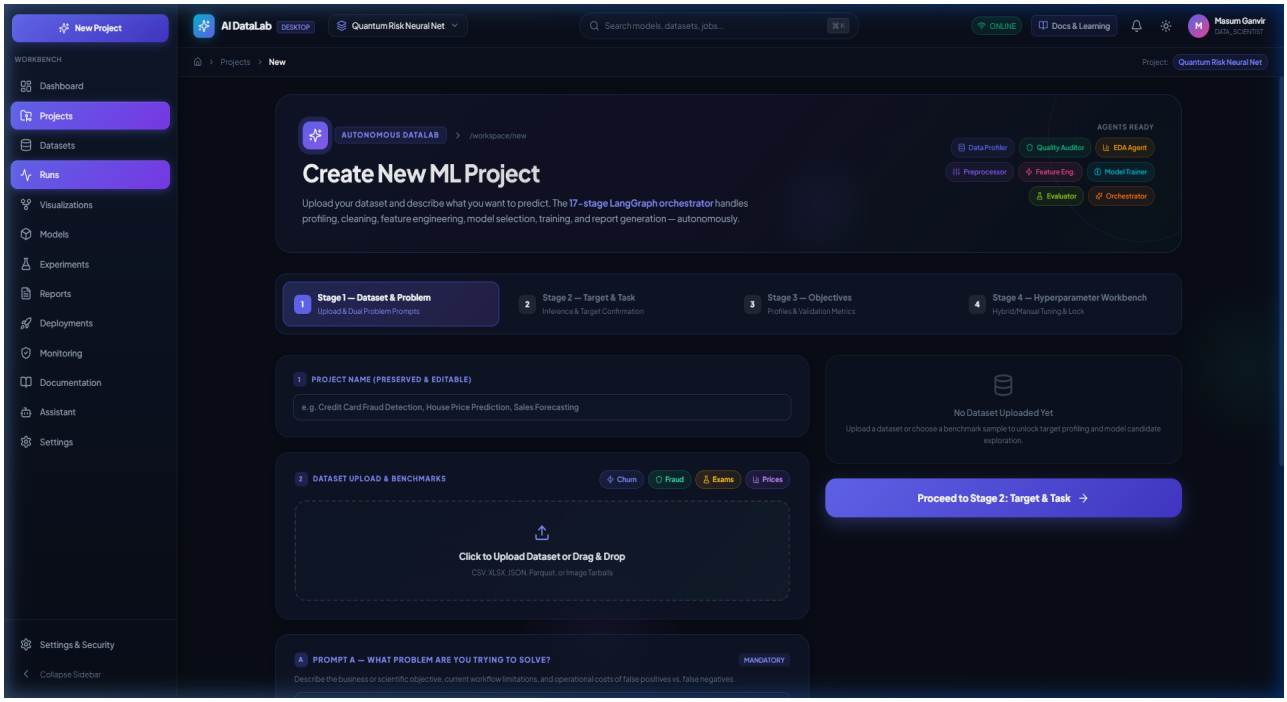


Figure: Stage 1: Two-Prompt Problem Definition & Natural Language Intent Capture

5. Multi-Agent Orchestration & State Machine

The agentic engine comprises 16 numbered specialized agents operating over a shared typed state graph:

ID	Agent Name	Primary Operational Responsibility	Verification
00	Orchestrator Agent	State transition control, stage gating, human approval loops	IMPLEMENTED
01	Dataset Profiling Agent	Schema detection, statistical moments, memory optimization	TESTED
02	Data Quality Agent	Missingness mechanisms (MCAR/MAR), cardinality, duplicates	TESTED
03	EDA Visualization Agent	Histogram distributions, correlation matrices, box plots	IMPLEMENTED
04	Outlier Intelligence Agent	IQR, Isolation Forest, Z-score analysis (flag vs filter)	TESTED
05	Missing Value Agent	KNN, Iterative, median, and mode imputation strategies	TESTED
06	Preprocessing Agent	ColumnTransformer synthesis, OneHot, TargetEncoding, Scaling	TESTED
07	Feature Engineering Agent	Polynomial interactions, binning, datetime extraction	TESTED
08	Feature Selection Agent	Mutual information, tree importance, VIF multicollinearity	TESTED
09	Data Leakage Agent	Strict cross-validation boundary verification, target leakage	TESTED
10	ML Strategy Agent	Candidate algorithm portfolio matching dataset constraints	TESTED
11	Model Training Agent	Stratified/k-fold training across candidate algorithms	TESTED
12	Model Tuning Agent	Optuna Bayesian hyperparameter search with pruning	TESTED
13	Model Evaluation Agent	Held-out test scoring, ROC-AUC, PR curves, confusion matrix	TESTED
14	Deployment Artifact Agent	Export standalone Python scripts, pickle models, manifests	TESTED
15	Reporting Notebook Agent	Executable Jupyter .ipynb generation and executive summaries	TESTED

6. Machine Learning Methodology & Leakage Prevention

A foundational guarantee of DataWise AI is absolute data leakage immunity. The platform enforces strict mathematical isolation between training, validation, and test folds:

1. **Strict Split Ordering:** Datasets are partitioned into Train (70%), Validation (15%), and Test (15%) splits prior to computing any statistical metric (mean, median, frequency, encoding mappings).
2. **Pipeline Encapsulation:** All feature transformations are encapsulated inside `sklearn.compose.ColumnTransformer` and `sklearn.pipeline.Pipeline`. Transformers are fitted strictly on `X_train` and evaluated via `transform()` on validation and test partitions.
3. **Target Leakage Auditing:** The Data Leakage Agent inspects pairwise feature-target mutual information and correlation scores. Features exhibiting greater than 0.98 predictive correlation are flagged for user confirmation to prevent accidental target duplication or proxy leakage.

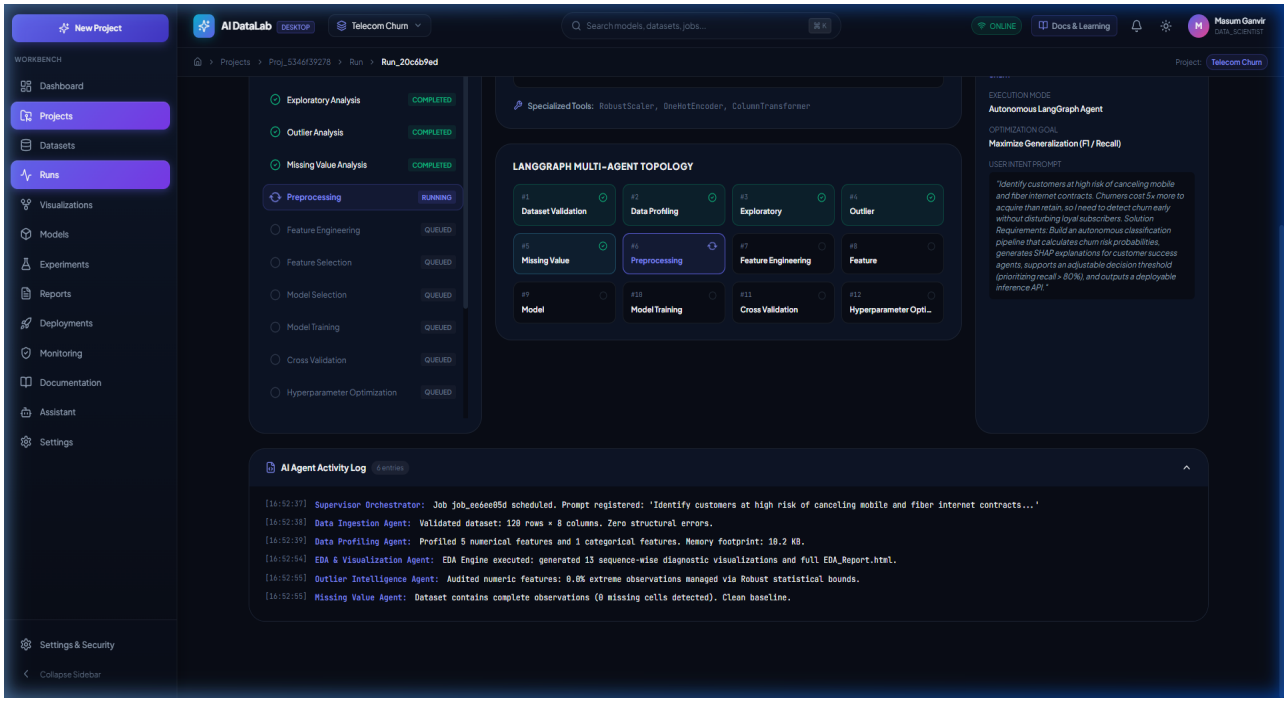


Figure: Real-Time Multi-Agent Autonomous Execution Telemetry & Audit Stream

7. Adaptive Algorithm Selection and Optuna Optimization

Rather than brute-forcing high-dimensional model grids, DataWise AI matches dataset attributes (number of rows, feature dimensionality, sparsity ratio, and target distribution) to algorithmic suitability scores:

- **Tabular Linear Regimes:** Ridge, Lasso, Logistic Regression, ElasticNet for high-interpretability or linearly separable benchmarks.
- **Tree Ensemble Regimes:** Random Forest, Extra Trees, XGBoost, LightGBM for complex non-linear boundary discovery and tabular dominance.
- **Optuna Hyperparameter Tuning:** Employs Tree-structured Parzen Estimator (TPE) Bayesian optimization with MedianPruner early stopping. Trials are bounded by wall-clock time and iteration caps.

8. Security Architecture and Project Isolation

DataWise AI enforces defense-in-depth across the web application and execution environments:

- **Authentication:** Multi-provider support via Clerk OAuth, Supabase JWT, and hardened local JWT tokens with SHA-256 password hashing.
- **Tenant & Project Isolation:** All database queries enforce strict user_id and project_id predicates. Artifact storage prefixes are isolated per project.
- **File Upload Security:** Magic byte inspection, MIME type validation, file size enforcement (100MB ceiling), and filename sanitization against path traversal.
- **Prompt Injection Defenses:** Structured prompt templating with system isolation tags prevents uploaded dataset strings from hijacking agent instructions.

9. Academic Citations & Related Work

Ref	Bibliographic Citation
[1]	Feurer, M., et al. 'Efficient and Robust Automated Machine Learning.' NeurIPS 2015.
[2]	Akiba, T., et al. 'Optuna: A Next-generation Hyperparameter Optimization Framework.' ACM KDD 2019.
[3]	Chen, T., & Guestrin, C. 'XGBoost: A Scalable Tree Boosting System.' ACM KDD 2016.
[4]	Ke, G., et al. 'LightGBM: A Highly Efficient Gradient Boosting Decision Tree.' NeurIPS 2017.
[5]	Pedregosa, F., et al. 'Scikit-learn: Machine Learning in Python.' JMLR 2011.
[6]	LangChain AI. 'LangGraph: Multi-Agent Workflows with Cycles and State.' 2024.